JobShield: Sequential Multi-Agent Execution Flow

Dedicated Pipeline Dossier: Clean sequential execution chain from initial user input down to final verdict arbiter and UI rendering.

1. Sequential Execution Table (Step-by-Step Chain)

Every agent and component executes in a defined, linear downward order:

Step 0	**Input Guardrails** guardrails.py	Pre-flight safety shield: enforces 30–25k character bounds, sanitizes null bytes, checks prompt-injection payloads, and filters toxic harassment.	Raw user text	**CrewAI Flow** (Step 1)	**IF Valid (≥30 chars)** ELSE Halt & Alert
Step 1	**CrewAI Flow State** pipeline.py	State container: initializes shared Pydantic ReviewState, creates run UUID, records start timestamp, and connects tracing callbacks.	Clean text string	**Agent 1: Parser** (Step 2)	Passes raw text in shared ReviewState
Step 2	**Agent 1: Parser** parser.py	Entity extraction specialist: uses regex/NLP to extract Company Name, Role Title, Contact Email/WhatsApp, Salary (LPA/CTC), and Experience (Years).	ReviewState text	**Agent 2: Coordinator** (Step 3)	Passes structured ParsedPosting object
Step 3	**Agent 2: Coordinator** pipeline.py	Pipeline orchestrator: evaluates extracted fields, plans downstream verification routes, and dispatches job text to threat scanners.	ParsedPosting fields	**Agent 3: Red Flags** (Step 4)	Dispatches text to deterministic rule engine
Step 4	**Agent 3: Red Flags** red_flags.py	Deterministic threat hunter: executes global regex (finditer) across 9 threat families (Upfront Fees, Sensitive Info, Telegram VIP, Urgency, etc.).	Job text + contacts	**Agent 4: FAISS Vector** (Step 5)	Passes list[Evidence] (Strong/Mod/Weak)
Step 5	**Agent 4: FAISS Vector** pipeline.py	Historical fraud cross-referencer: embeds text via all-MiniLM-L6-v2 (384-d) and queries FAISS index of 18,000+ historical labeled scams.	Job description text	**Agent 5: Verifier** (Step 6)	Passes similarity score & nearest scam neighbors
Step 6	**Agent 5: Verifier** mcp_server.py	Network verification: FastMCP DNS A/AAAA lookup, ICANN RDAP domain registration age query, and Live Web Search fallback with callback loop.	Target company/domain	**Agent 6: Arbiter** (Step 7)	**CALLBACK:** If RDAP fails → retry Web Search. Passes CompanyCheck
Step 7	**Agent 6: Arbiter** policy.py	Deterministic decision arbiter: evaluates conditional decision tree across all collected evidence, computing final category & 0–100 risk score.	All agent evidence	**Pydantic Output** (Step 8)	• Fee/OTP → **HIGH RISK** (50–100) • Ambiguous → **NEEDS VERIF.** (20–45) • Clean → **LOW RISK** (0–15)
Step 8	**Storage & UI** storage.py / app.py	Persistent audit & presentation: saves record in SQLite database, updates event audit trace, and renders live Streamlit verdict dashboard.	AnalysisResult	**End of Pipeline**	Renders live result on user browser screen

2. Downward Sequential Flowchart (Top-to-Bottom Flow)

Every agent calls the next in sequence with explicit arrow conditions:

